



Proyecto 2:
Diseño de proceso de ETL y configurar cargas incrementales .

Estudiantes:

Keylor Herrera Fuentes	2021046723
Sebastián Castillo Monestel	2024070857
Fiorella González López	2024089144

Almacenes de Datos y Procesamiento OLAP
GR 20

Profesor:

Prof. Alberto Shum Chan

Fecha de entrega: 27/05/2026

Semestre I, 2026

Objetivo

Aprender a crear un proceso de ETL usando un proyecto de Visual Studio.
Investigar y documentar
el proceso para configurar cargas incrementales.

Requerimientos

1. Con base en el proyecto 1, diseñe las tablas intermedias (staging) necesarias para almacenar la información proveniente de la base de datos transaccional (OLTP) que será utilizada en su almacén de datos. Estas tablas deben crearse en la base de datos del almacén de datos (DW). Proveer los scripts. 10 pts.

Con base en el modelo transaccional del Proyecto 1 y la base de datos CatwalkGlowing_OLTP, se diseñaron tablas intermedias o staging dentro de la base de datos del almacén de datos CatwalkGlowing_DW. Estas tablas tienen como objetivo recibir temporalmente la información proveniente del OLTP antes de aplicar las transformaciones necesarias para cargar las dimensiones y la tabla de hechos del modelo multidimensional. El Proyecto 2 solicita que estas tablas staging se creen en la base DW y que almacenen la información usada posteriormente en el proceso ETL.

Las tablas staging propuestas son: StgCliente, StgProducto, StgPromocion, StgTiempo y StgVentaDetalle. Estas tablas permiten separar la extracción inicial desde el OLTP de la etapa de transformación y carga hacia el modelo dimensional. Además, se consideran las correcciones del profesor, evitando llevar al DW atributos operacionales innecesarios como teléfono, email, dirección exacta, stock o precios fijos del producto.

Script SQL: creación de tablas staging

```
USE CatwalkGlowing_DW;  
GO
```

```
-- =====  
-- LIMPIEZA PREVIA SI LAS TABLAS YA EXISTEN  
-- =====
```

```
IF OBJECT_ID('dbo.StgVentaDetalle', 'U') IS NOT NULL  
    DROP TABLE dbo.StgVentaDetalle;  
GO
```

```
IF OBJECT_ID('dbo.StgTipoDePrecio', 'U') IS NOT NULL  
    DROP TABLE dbo.StgTipoDePrecio;  
GO
```

```
IF OBJECT_ID('dbo.StgTiempo', 'U') IS NOT NULL
    DROP TABLE dbo.StgTiempo;
GO
```

```
IF OBJECT_ID('dbo.StgPromocion', 'U') IS NOT NULL
    DROP TABLE dbo.StgPromocion;
GO
```

```
IF OBJECT_ID('dbo.StgProducto', 'U') IS NOT NULL
    DROP TABLE dbo.StgProducto;
GO
```

```
IF OBJECT_ID('dbo.StgCliente', 'U') IS NOT NULL
    DROP TABLE dbo.StgCliente;
GO
```

```
-- =====
-- STAGING DE CLIENTES
-- Fuente OLTP:
--   Cliente, Canton, Provincia
-- Destino final:
--   DimCliente
-- =====
```

```
CREATE TABLE dbo.StgCliente (
    id_cliente INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    primer_apellido VARCHAR(100) NOT NULL,
    segundo_apellido VARCHAR(100) NULL,
    nombre_completo VARCHAR(300) NULL,
    canton VARCHAR(100) NOT NULL,
    provincia VARCHAR(100) NOT NULL,
    fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
);
GO
```

```
-- =====
-- STAGING DE PRODUCTOS
-- Fuente OLTP:
--   Producto, Categoria, Marca
-- Carga: CDC incremental
```

```

-- Destino final:
-- DimProducto
-- =====

CREATE TABLE dbo.StgProducto (
    id_producto INT NOT NULL,
    nombre_producto VARCHAR(100) NOT NULL,
    categoria VARCHAR(100) NOT NULL,
    marca VARCHAR(100) NOT NULL,
    fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
);
GO

```

```

-- =====
-- STAGING DE PROMOCIONES
-- Fuente OLTP:
-- Promocion
-- Destino final:
-- DimPromocion
-- =====

```

```

CREATE TABLE dbo.StgPromocion (
    id_promocion INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    tipo VARCHAR(50) NOT NULL,
    descripcion VARCHAR(255) NULL,
    fecha_inicio DATE NULL,
    fecha_fin DATE NULL,
    fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
);
GO

```

```

-- =====
-- STAGING DE TIEMPO
-- Fuente OLTP:
-- Venta.fecha
-- Destino final:
-- DimTiempo
-- =====

```

```

CREATE TABLE dbo.StgTiempo (
    id_tiempo INT NOT NULL,

```

```

    fecha DATE NOT NULL,
    dia INT NOT NULL,
    mes INT NOT NULL,
    nombre_mes VARCHAR(20) NOT NULL,
    trimestre INT NOT NULL,
    anio INT NOT NULL,
    nombre_dia VARCHAR(20) NOT NULL,
    fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
);
GO

```

```

-- =====
-- STAGING DE TIPO DE PRECIO
-- Fuente OLTP:
-- Producto.precio_detalle, Producto.precio_mayorista
-- Destino final:
-- DimTipoDePrecio
-- =====

```

```

CREATE TABLE dbo.StgTipoDePrecio (
    id_tipo_de_precio INT NOT NULL,
    precio_detalle DECIMAL(10,2) NOT NULL,
    precio_mayorista DECIMAL(10,2) NULL,
    fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
);
GO

```

```

-- =====
-- STAGING DE VENTAS Y DETALLE
-- Fuente OLTP:
-- Venta, DetalleVenta, Producto (para tipo de precio)
-- Carga: CDC incremental
-- Destino final:
-- FactVenta
-- =====

```

```

CREATE TABLE dbo.StgVentaDetalle (
    id_venta INT NOT NULL,
    id_detalle INT NOT NULL,
    fecha DATE NOT NULL,
    id_cliente INT NOT NULL,
    id_producto INT NOT NULL,

```

```
id_promocion INT NULL,  
id_tipo_de_precio INT NOT NULL,  
cantidad_vendida INT NOT NULL,  
precio_aplicado DECIMAL(10,2) NOT NULL,  
monto_total DECIMAL(10,2) NOT NULL,  
tipo_precio_aplicado VARCHAR(50) NOT NULL,  
fecha_carga DATETIME NOT NULL DEFAULT GETDATE()  
);  
GO
```

```
CREATE INDEX IX_StgCliente_id_cliente  
ON dbo.StgCliente(id_cliente);  
GO
```

```
CREATE INDEX IX_StgProducto_id_producto  
ON dbo.StgProducto(id_producto);  
GO
```

```
CREATE INDEX IX_StgPromocion_id_promocion  
ON dbo.StgPromocion(id_promocion);  
GO
```

```
CREATE INDEX IX_StgTiempo_fecha  
ON dbo.StgTiempo(fecha);  
GO
```

```
CREATE INDEX IX_StgTipoDePrecio_id  
ON dbo.StgTipoDePrecio(id_tipo_de_precio);  
GO
```

```
CREATE INDEX IX_StgVentaDetalle_id_venta  
ON dbo.StgVentaDetalle(id_venta);  
GO
```

```
CREATE INDEX IX_StgVentaDetalle_id_cliente  
ON dbo.StgVentaDetalle(id_cliente);  
GO
```

```
CREATE INDEX IX_StgVentaDetalle_id_producto  
ON dbo.StgVentaDetalle(id_producto);  
GO
```

Las tablas staging fueron diseñadas según las dimensiones y hechos que se utilizarán en el almacén de datos. StgCliente, StgProducto, StgTipoDePrecio, StgPromocion y StgTiempo servirán como base para cargar las dimensiones, mientras que StgVentaDetalle almacenará la información necesaria para cargar la tabla de hechos de ventas. Estas tablas se ubican en CatwalkGlowing_DW, ya que funcionan como área intermedia del proceso ETL antes de cargar el modelo multidimensional final.

2. Investigue sobre la funcionalidad Change Data Capture (CDC) SQL Server y documente los pasos necesarios para configurarla en su base de datos OLTP, de modo que esto le permita realizar cargas incrementales en el proceso de ETL. Para efectos de este proyecto sólo nos interesa los datos nuevos (INSERT), no se considerarán los borrados o modificaciones. Configurar CDC en una tabla transaccional y en una tabla de catálogo. Proveer los scripts y capturas de pantalla del proceso de configuración y pruebas. 40 pts.

Para implementar cargas incrementales en el proceso ETL, se investigó y configuró la funcionalidad Change Data Capture (CDC) de SQL Server. Esta funcionalidad permite registrar los cambios realizados sobre tablas específicas de una base de datos, almacenando la información en tablas internas dentro del esquema cdc.

En este proyecto, CDC se configuró sobre la base transaccional CatwalkGlowing_OLTP, ya que esta representa el origen de datos operacional. Se habilitó CDC en una tabla transaccional, Venta, y en una tabla de catálogo, Producto, cumpliendo con lo solicitado en el enunciado del proyecto. Para efectos de este trabajo solo se consideran los datos nuevos insertados, por lo que en las consultas CDC se filtrará la operación correspondiente a INSERT.

Script 1: habilitar CDC en la base OLTP

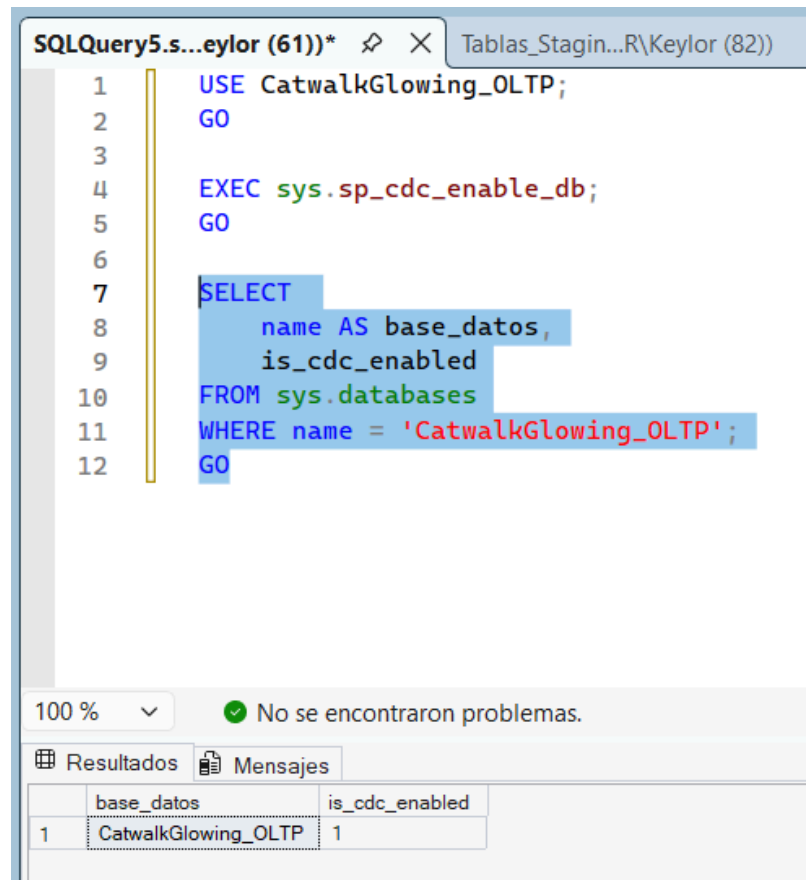
```
USE CatwalkGlowing_OLTP;  
GO
```

```
EXEC sys.sp_cdc_enable_db;  
GO
```

Verificar que CDC quedó activo en la base

```
SELECT  
    name AS base_datos,  
    is_cdc_enabled
```

```
FROM sys.databases
WHERE name = 'CatwalkGlowing_OLTP';
GO
```

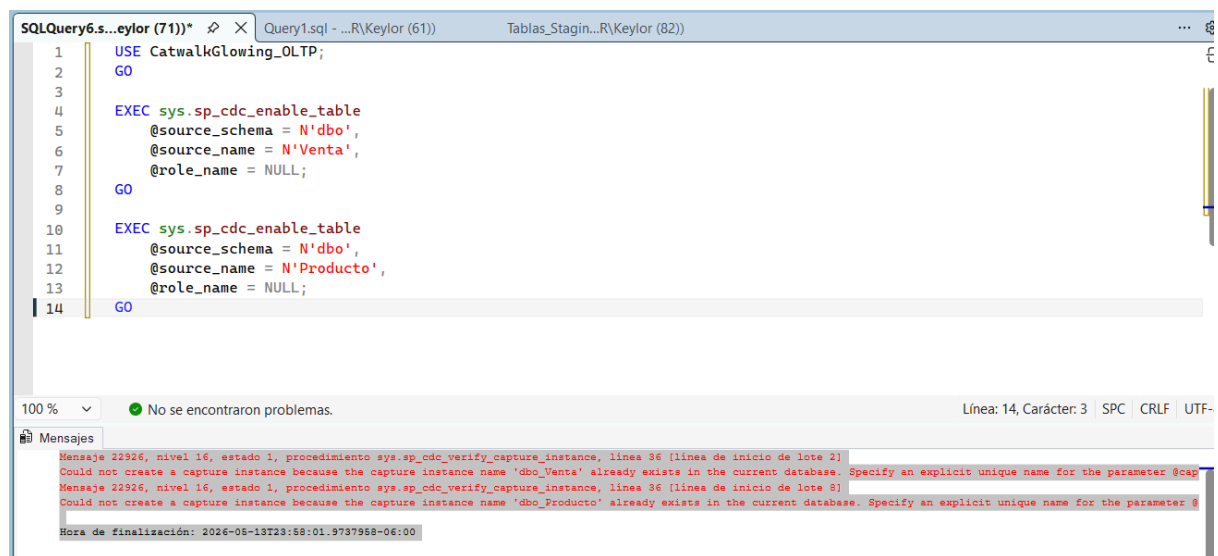


Script 2: habilitar CDC en tabla transaccional y catálogo

```
USE CatwalkGlowing_OLTP;
GO
```

```
EXEC sys.sp_cdc_enable_table
    @source_schema = N'dbo',
    @source_name = N'Venta',
    @role_name = NULL;
GO
```

```
EXEC sys.sp_cdc_enable_table
    @source_schema = N'dbo',
    @source_name = N'Producto',
    @role_name = NULL;
GO
```



```
1  USE CatwalkGlowing_OLTP;
2  GO
3
4  EXEC sys.sp_cdc_enable_table
5      @source_schema = N'dbo',
6      @source_name = N'Venta',
7      @role_name = NULL;
8  GO
9
10 EXEC sys.sp_cdc_enable_table
11     @source_schema = N'dbo',
12     @source_name = N'Producto',
13     @role_name = NULL;
14 GO
```

100 % No se encontraron problemas. Línea: 14, Carácter: 3 SPC CRLF UTF-

Mensajes

Mensaje 22926, nivel 16, estado 1, procedimiento sys.sp_cdc_verify_capture_instance, línea 36 [línea de inicio de lote 2]
Could not create a capture instance because the capture instance name 'dbo.Venta' already exists in the current database. Specify an explicit unique name for the parameter @cap

Mensaje 22926, nivel 16, estado 1, procedimiento sys.sp_cdc_verify_capture_instance, línea 36 [línea de inicio de lote 8]
Could not create a capture instance because the capture instance name 'dbo.Producto' already exists in the current database. Specify an explicit unique name for the parameter @

Hora de finalización: 2026-05-13T23:58:01.9737958-06:00

Este error es algo bueno ya que significa algo bueno: CDC ya está habilitado para Venta y Producto.

Script 3: verificar tablas con CDC activo

```
USE CatwalkGlowing_OLTP;
GO
```

```
SELECT
    name AS tabla,
    is_tracked_by_cdc
FROM sys.tables
WHERE name IN ('Venta', 'Producto');
GO
```

SQLQuery7.s...eylor (65))*

```
1  USE CatwalkGlowing_OLTP;
2  GO
3
4  SELECT
5      name AS tabla,
6      is_tracked_by_cdc
7  FROM sys.tables
8  WHERE name IN ('Venta', 'Producto');
9  GO
```

100 % No se encontraron problemas.

Resultados Mensajes

	tabla	is_tracked_by_cdc
1	Producto	1
2	Venta	1

Script 4: revisar las tablas CDC creadas

```
USE CatwalkGlowing_OLTP;
GO
```

```
SELECT
    name AS tabla_cdc
FROM sys.tables
WHERE schema_id = SCHEMA_ID('cdc');
GO
```

The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are two tabs: 'SQLQuery8.s...eylor (56))' and 'Query3.sql - ...R\Keylor (65))'. The active query window contains the following SQL script:

```
1  USE CatwalkGlowing_OLTP;
2  GO
3
4  SELECT
5      name AS tabla_cdc
6  FROM sys.tables
7  WHERE schema_id = SCHEMA_ID('cdc');
8  GO
```

Below the query window, there is a status bar showing '100 %' and a green checkmark with the text 'No se encontraron problemas.' Below this, there are two tabs: 'Resultados' and 'Mensajes'. The 'Resultados' tab is active, displaying a table with the results of the query:

	tabla_cdc
1	change_tables
2	ddl_history
3	lsn_time_mapping
4	captured_columns
5	index_columns
6	dbo_Venta_CT
7	dbo_Producto_CT

Las tablas CDC creadas son dbo_Venta_CT y dbo_Producto_CT.

Script 5: insertar datos nuevos para probar CDC

Insertar nuevo producto

```
USE CatwalkGlowing_OLTP;
GO
```

```
INSERT INTO Producto (
    id_producto,
```

```
nombre,  
stock_disponible,  
precio_detalle,  
precio_mayorista,  
id_categoria,  
id_marca  
)  
VALUES (  
21,  
'Tinta Labial Velvet',  
90,  
4900.00,  
4200.00,  
9,  
3  
);  
GO
```

Insertar nueva venta

```
USE CatwalkGlowing_OLTP;  
GO
```

```
INSERT INTO Venta (  
id_venta,  
fecha,  
mensaje,  
precio_final,  
id_cliente  
)  
VALUES (  
501,  
'2026-04-15',  
'Venta incremental de prueba CDC',  
9800.00,  
1  
);  
GO
```

Insertar detalle para esa venta

```
INSERT INTO DetalleVenta (  
id_detalle,  
id_venta,  
id_producto,
```

```

cantidad,
precio_venta_unidad,
tipo_precio,
id_promocion
)
VALUES (
    1001,
    501,
    21,
    2,
    4900.00,
    'detalle',
    NULL
);
GO

```

Script 6: consultar cambios capturados por CDC, consultas directamente de las tablas CDC

En CDC, el valor `__$operation = 2` representa un INSERT. Por eso se filtra ese valor para tomar solo registros nuevos.

Ver nuevos productos capturados

```

USE CatwalkGlowing_OLTP;
GO

```

```

SELECT *
FROM cdc.dbo_Producto_CT
WHERE __$operation = 2;
GO

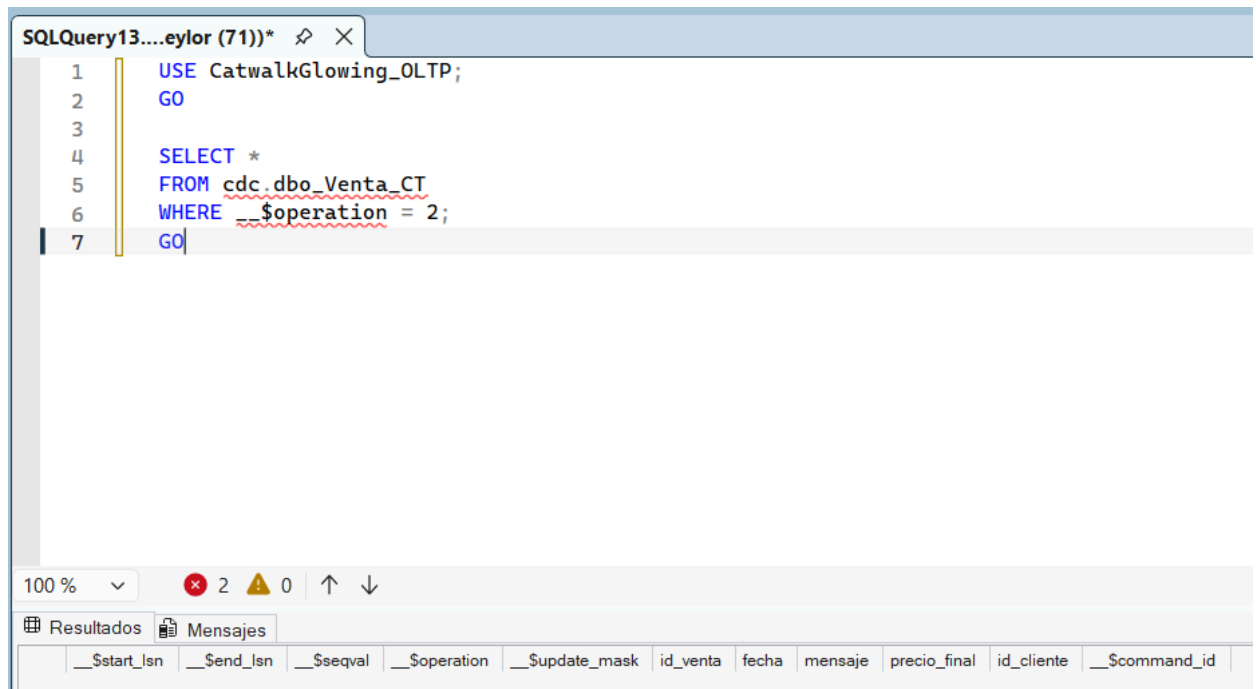
```



Ver nuevas ventas capturadas

```
USE CatwalkGlowing_OLTP;  
GO
```

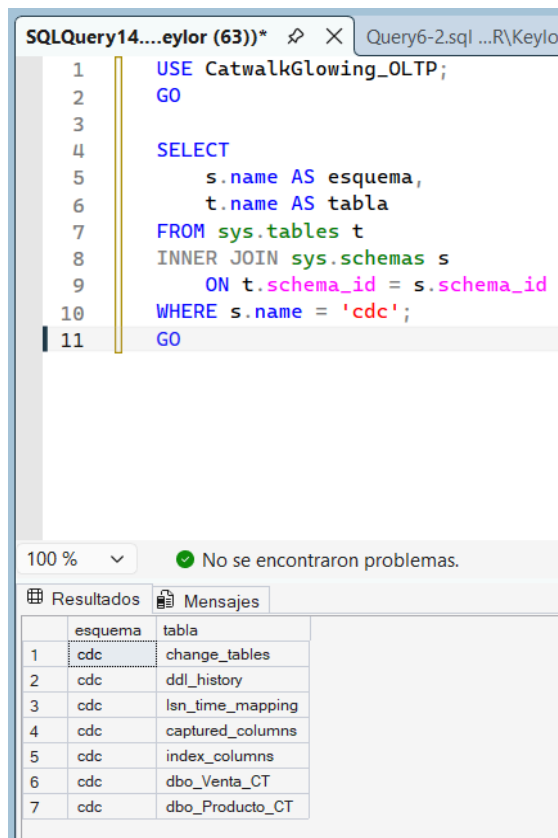
```
SELECT *  
FROM cdc.dbo_Venta_CT  
WHERE __$operation = 2;  
GO
```



Nombre exacto de las tablas CDC:

```
USE CatwalkGlowing_OLTP;  
GO
```

```
SELECT  
    s.name AS esquema,  
    t.name AS tabla  
FROM sys.tables t  
INNER JOIN sys.schemas s  
    ON t.schema_id = s.schema_id  
WHERE s.name = 'cdc';  
GO
```



The screenshot shows a SQL Server Enterprise Manager interface. At the top, a query window titled 'SQLQuery14....eylor (63))' contains the following T-SQL code:

```
1  USE CatwalkGlowing_OLTP;
2  GO
3
4  SELECT
5      s.name AS esquema,
6      t.name AS tabla
7  FROM sys.tables t
8  INNER JOIN sys.schemas s
9      ON t.schema_id = s.schema_id
10 WHERE s.name = 'cdc';
11 GO
```

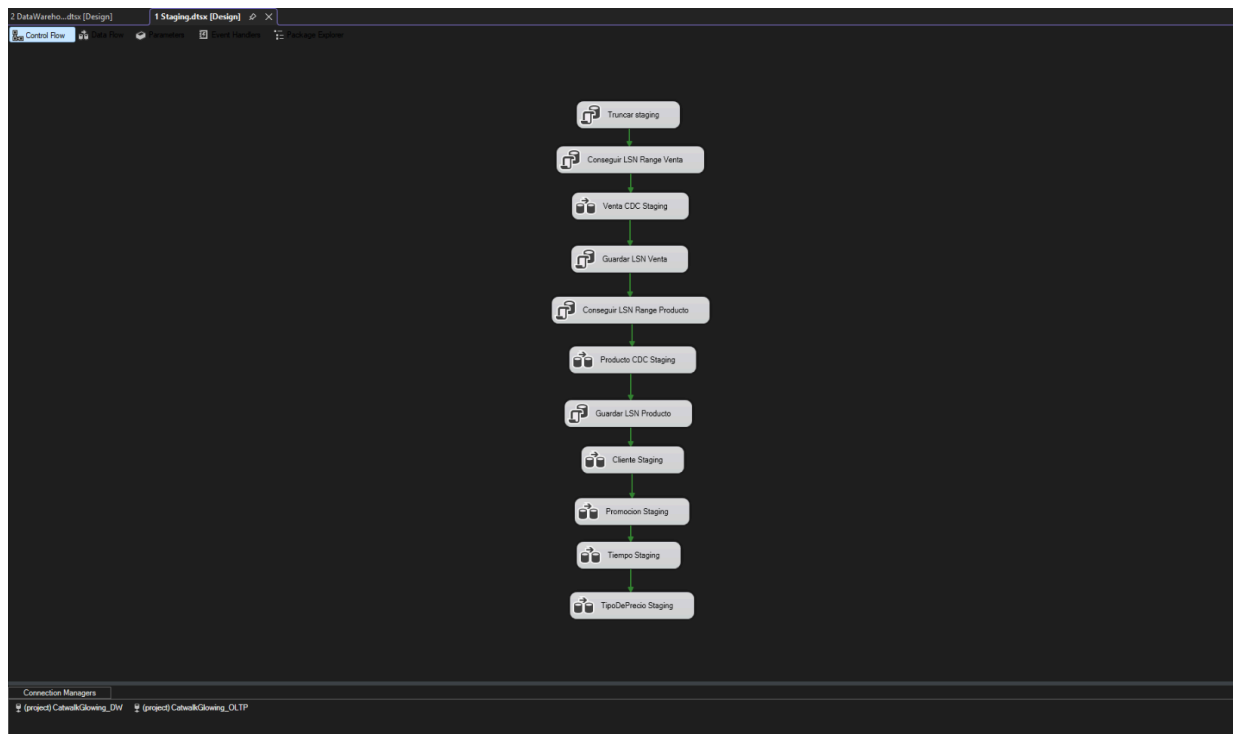
Below the query window, a status bar indicates '100 %' zoom and 'No se encontraron problemas.' (No problems found). Below that, the 'Resultados' (Results) tab is active, displaying a table with two columns: 'esquema' (schema) and 'tabla' (table). The table contains seven rows of data:

	esquema	tabla
1	cdc	change_tables
2	cdc	ddl_history
3	cdc	lsn_time_mapping
4	cdc	captured_columns
5	cdc	index_columns
6	cdc	dbo_Venta_CT
7	cdc	dbo_Producto_CT

Con esta configuración, la base CatwalkGlowing_OLTP queda preparada para realizar cargas incrementales hacia el proceso ETL. La tabla Venta permite capturar nuevos registros transaccionales, mientras que la tabla Producto permite capturar nuevos registros de catálogo. Para este proyecto únicamente se consideran las operaciones de inserción, por lo que las consultas CDC filtran los cambios con `__$operation = 2`, correspondiente a registros nuevos.

3. Construya un proyecto en Visual Studio para realizar el proceso de ETL que contenga los siguientes paquetes (DTS):
 - a. Paquete para cargar de las tablas transaccionales (OLTP) a las tablas staging utilizando la funcionalidad del CDC para implementar cargas incrementales. Se recomienda que como parte del proceso de carga de las tablas staging estás se limpien (truncate) antes de cada carga. 25 pts.

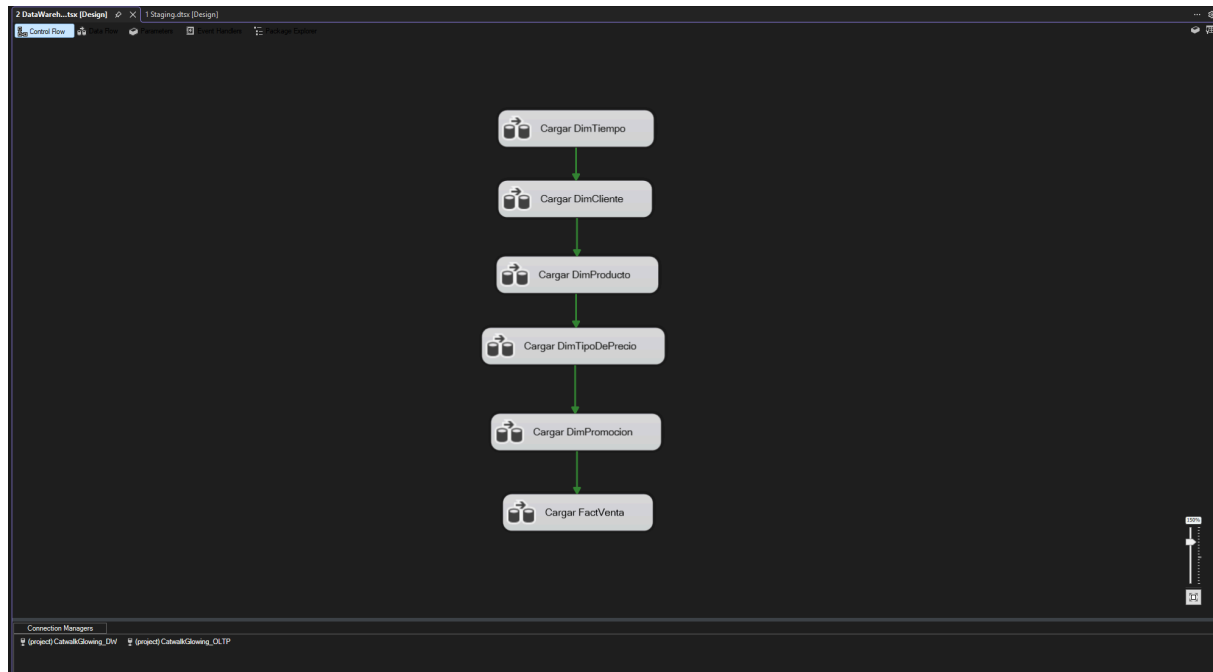
Se elaboró el paquete de staging de la siguiente manera:



Primero, se truncan las tablas de staging antes de trabajar en estas. Después, se trabaja en el staging de cada tabla en este orden: VentaDetalle, Producto, Cliente, Promocion, Tiempo y TipoDePrecio. VentaDetalle y Producto se hacen con la funcionalidad CDC implementada en el punto previo. En estas se consigue el rango de LSN a trabajar utilizando la tabla cdc_state antes de realizar el staging y luego se guarda la última posición de los logs que fue procesada en esta misma tabla. Al finalizar la ejecución de este paquete, las tablas de staging quedan pobladas y la base queda lista para ejecutar el paquete de inserción a las dimensiones y tabla de hechos.

- b. Paquete que realice las transformaciones y limpieza de datos y que luego haga la carga de las tablas staging (procesadas) a las tablas del modelo multidimensional (dimensiones y hechos). 25 pts.

Se elaboró el paquete de Data Warehouse de la siguiente manera:



En este, se limpian las entradas y se cargan a las dimensiones todas las entradas nuevas que se vean en las tablas de staging en el siguiente orden: DimTiempo, DimCliente, DimProducto, DimTipoDePrecio y DimPromocion. Luego, se realiza la carga de las entradas en el staging a FactVenta, donde se asocia cada columna del staging de DetalleVenta a una dimensión y se agrega a la tabla de hechos. Después de la ejecución de este paquete, queda poblada la tabla de hechos en la base de datos.

1 | select * from dbo.factventa;

100 % No issues found Ln: 1, Ch: 29 TABS CRLF UTF-8

	id_fact	id_tiempo	id_cliente	id_producto	id_promocion	id_tipo_de_precio	cantidad_vendida	monto_total	tipo_precio_aplicado
1	1	20240102	1	1	0	15	1	5600.00	mayorista
2	2	20240102	1	7	0	7	3	12900.00	mayorista
3	3	20240103	2	2	0	12	2	10200.00	mayorista
4	4	20240103	2	8	6	4	1	4230.00	detalle
5	5	20240104	3	3	3	5	3	12960.00	detalle
6	6	20240104	3	9	0	10	2	11000.00	detalle
7	7	20240105	4	4	0	9	4	20800.00	detalle
8	8	20240105	4	10	0	8	3	13200.00	mayorista
9	9	20240106	5	5	0	2	1	3700.00	mayorista
10	10	20240106	5	11	0	21	1	8100.00	mayorista
11	11	20240107	6	6	0	17	2	13800.00	mayorista
12	12	20240107	6	12	10	1	2	4500.00	detalle
13	13	20240108	7	7	7	7	3	13500.00	detalle
14	14	20240108	7	13	0	14	3	19200.00	detalle
15	15	20240109	8	8	0	4	4	18800.00	detalle
16	16	20240109	8	14	0	16	1	6200.00	mayorista
17	17	20240110	9	9	0	10	1	4700.00	mayorista
18	18	20240110	9	15	0	20	2	15600.00	mayorista
19	19	20240111	10	10	0	8	2	8800.00	mayorista
20	20	20240111	10	16	14	18	3	22410.00	detalle
21	21	20240112	11	11	11	21	3	24840.00	detalle
22	22	20240112	11	17	0	13	1	6200.00	detalle

Query executed successfully. | localhost (15.0 RTM) | GUEGUE\sebeca (58) | CatwalkGlowing_DW | 00:00:00 | Row: 1, Col: 1 | 1,000 rows

Script 7: creación base OLTP (CDC en Producto y Venta incluido)

-- Creación de modelo OLTP para Catwalk Glowing

```
CREATE DATABASE CatwalkGlowing_OLTP;
GO
```

```
USE CatwalkGlowing_OLTP;
GO
```

```
CREATE TABLE Provincia (
    id_provincia INT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);
GO
```

```
CREATE TABLE Canton (
    id_canton INT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    id_provincia INT NOT NULL,
    CONSTRAINT FK_Canton_Provincia
        FOREIGN KEY (id_provincia) REFERENCES Provincia(id_provincia)
);
GO
```

```

CREATE TABLE Cliente (
    id_cliente INT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    primer_apellido VARCHAR(100) NOT NULL,
    segundo_apellido VARCHAR(100) NULL,
    telefono VARCHAR(20) NULL,
    email VARCHAR(100) NULL,
    direccion_exacta VARCHAR(255) NULL,
    codigo_mayorista VARCHAR(50) NULL,
    es_mayorista BIT NOT NULL DEFAULT 0,
    id_canton INT NOT NULL,
    CONSTRAINT FK_Cliente_Canton
        FOREIGN KEY (id_canton) REFERENCES Canton(id_canton),
    CONSTRAINT UQ_Cliente_Email UNIQUE (email)
);
GO

```

```

CREATE TABLE Categoria (
    id_categoria INT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);
GO

```

```

CREATE TABLE Marca (
    id_marca INT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);
GO

```

```

CREATE TABLE Producto (
    id_producto INT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    stock_disponible INT NOT NULL,
    precio_detalle DECIMAL(10,2) NOT NULL,
    precio_mayorista DECIMAL(10,2) NULL,
    id_categoria INT NOT NULL,
    id_marca INT NOT NULL,
    CONSTRAINT FK_Producto_Categoria
        FOREIGN KEY (id_categoria) REFERENCES Categoria(id_categoria),
    CONSTRAINT FK_Producto_Marca
        FOREIGN KEY (id_marca) REFERENCES Marca(id_marca),
    CONSTRAINT CHK_Producto_Stock
        CHECK (stock_disponible >= 0),
    CONSTRAINT CHK_Producto_PrecioDetalle

```

```

        CHECK (precio_detalle > 0),
    CONSTRAINT CHK_Producto_PrecioMayorista
        CHECK (precio_mayorista IS NULL OR precio_mayorista > 0)
);
GO

```

```

CREATE TABLE Promocion (
    id_promocion INT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    tipo VARCHAR(20) NOT NULL,
    descripcion VARCHAR(255) NULL,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE NOT NULL,
    activa BIT NOT NULL DEFAULT 1,
    CONSTRAINT CHK_Promocion_Tipo
        CHECK (tipo IN ('precio_mayorista', 'combo', 'otro')),
    CONSTRAINT CHK_Promocion_Fechas
        CHECK (fecha_fin >= fecha_inicio)
);
GO

```

```

CREATE TABLE PromocionProducto (
    id_promocion INT NOT NULL,
    id_producto INT NOT NULL,
    PRIMARY KEY (id_promocion, id_producto),
    CONSTRAINT FK_PromocionProducto_Promocion
        FOREIGN KEY (id_promocion) REFERENCES Promocion(id_promocion),
    CONSTRAINT FK_PromocionProducto_Producto
        FOREIGN KEY (id_producto) REFERENCES Producto(id_producto)
);
GO

```

```

CREATE TABLE Venta (
    id_venta INT PRIMARY KEY,
    fecha DATE NOT NULL,
    mensaje VARCHAR(255) NULL,
    precio_final DECIMAL(10,2) NOT NULL,
    id_cliente INT NOT NULL,
    CONSTRAINT FK_Venta_Cliente
        FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente),
    CONSTRAINT CHK_Venta_PrecioFinal
        CHECK (precio_final >= 0)
);
GO

```

```
);  
GO
```

```
CREATE TABLE DetalleVenta (  
    id_detalle INT PRIMARY KEY,  
    id_venta INT NOT NULL,  
    id_producto INT NOT NULL,  
    cantidad INT NOT NULL,  
    precio_venta_unidad DECIMAL(10,2) NOT NULL,  
    tipo_precio VARCHAR(20) NOT NULL DEFAULT 'detalle',  
    id_promocion INT NULL,  
    CONSTRAINT FK_DetalleVenta_Venta  
        FOREIGN KEY (id_venta) REFERENCES Venta(id_venta),  
    CONSTRAINT FK_DetalleVenta_Producto  
        FOREIGN KEY (id_producto) REFERENCES Producto(id_producto),  
    CONSTRAINT FK_DetalleVenta_Promocion  
        FOREIGN KEY (id_promocion) REFERENCES Promocion(id_promocion),  
    CONSTRAINT CHK_DetalleVenta_Cantidad  
        CHECK (cantidad > 0),  
    CONSTRAINT CHK_DetalleVenta_Precio  
        CHECK (precio_venta_unidad >= 0),  
    CONSTRAINT CHK_DetalleVenta_TipoPrecio  
        CHECK (tipo_precio IN ('detalle', 'mayorista_volumen', 'mayorista_membresia',  
'promocion'))  
);  
GO
```

```
CREATE INDEX IX_Canton_id_provincia ON Canton(id_provincia);  
CREATE INDEX IX_Cliente_id_canton ON Cliente(id_canton);  
CREATE INDEX IX_Producto_id_categoria ON Producto(id_categoria);  
CREATE INDEX IX_Producto_id_marca ON Producto(id_marca);  
CREATE INDEX IX_PromocionProducto_id_producto ON  
PromocionProducto(id_producto);  
CREATE INDEX IX_Venta_id_cliente ON Venta(id_cliente);  
CREATE INDEX IX_DetalleVenta_id_venta ON DetalleVenta(id_venta);  
CREATE INDEX IX_DetalleVenta_id_producto ON DetalleVenta(id_producto);  
CREATE INDEX IX_DetalleVenta_id_promocion ON DetalleVenta(id_promocion);  
GO
```

```
EXEC sys.sp_cdc_enable_db;  
GO
```

```
EXEC sys.sp_cdc_enable_table  
    @source_schema = 'dbo',
```

```
@source_name = 'Venta',  
@role_name = NULL;  
GO
```

```
EXEC sys.sp_cdc_enable_table  
@source_schema = 'dbo',  
@source_name = 'Producto',  
@role_name = NULL;  
GO
```

```
-- Inserción de datos de catalogo  
-- PROVINCIAS  
INSERT INTO Provincia (id_provincia, nombre) VALUES  
(1, 'San José'),  
(2, 'Alajuela'),  
(3, 'Cartago'),  
(4, 'Heredia'),  
(5, 'Guanacaste'),  
(6, 'Puntarenas'),  
(7, 'Limón');  
GO
```

```
-- CANTONES  
INSERT INTO Canton (id_canton, nombre, id_provincia) VALUES  
(1, 'San José', 1),  
(2, 'Escazú', 1),  
(3, 'Desamparados', 1),  
(4, 'Alajuela', 2),  
(5, 'San Ramón', 2),  
(6, 'Grecia', 2),  
(7, 'Cartago', 3),  
(8, 'Paraíso', 3),  
(9, 'La Unión', 3),  
(10, 'Heredia', 4),  
(11, 'Barva', 4),  
(12, 'Santo Domingo', 4),  
(13, 'Liberia', 5),  
(14, 'Nicoya', 5),  
(15, 'Santa Cruz', 5),  
(16, 'Puntarenas', 6),  
(17, 'Esparza', 6),  
(18, 'Quepos', 6),
```

```
(19, 'Limón', 7),  
(20, 'Pococi', 7);  
GO
```

-- CATEGORIAS

```
INSERT INTO Categoria (id_categoria, nombre) VALUES  
(1, 'Bases'),  
(2, 'Correctores'),  
(3, 'Polvos'),  
(4, 'Rubores'),  
(5, 'Iluminadores'),  
(6, 'Sombras'),  
(7, 'Delineadores'),  
(8, 'Mascaras'),  
(9, 'Labiales'),  
(10, 'Gloss'),  
(11, 'Brochas'),  
(12, 'Esponjas'),  
(13, 'Cuidado facial'),  
(14, 'Cuidado corporal'),  
(15, 'Cuidado del cabello'),  
(16, 'Serums'),  
(17, 'Primer'),  
(18, 'Fijadores'),  
(19, 'Removedores'),  
(20, 'Accesorios');  
GO
```

-- MARCAS

```
INSERT INTO Marca (id_marca, nombre) VALUES  
(1, 'Maybelline'),  
(2, 'L'Oréal'),  
(3, 'NYX'),  
(4, 'Revlon'),  
(5, 'Milani'),  
(6, 'e.l.f.'),  
(7, 'Wet n Wild'),  
(8, 'Essence'),  
(9, 'Catrice'),  
(10, 'Sheglam'),  
(11, 'Beauty Creations'),  
(12, 'Physicians Formula'),  
(13, 'ColourPop'),  
(14, 'Morphe'),
```

```
(15, 'Rude'),
(16, 'Bissu'),
(17, 'Italia Deluxe'),
(18, 'Saniye'),
(19, 'AOA Studio'),
(20, 'Moirá');
GO
```

-- PRODUCTOS

```
INSERT INTO Producto (
    id_producto, nombre, stock_disponible, precio_detalle, precio_mayorista,
    id_categoria, id_marca
) VALUES
(1, 'Base Fit Me Matte', 150, 6500.00, 5600.00, 1, 1),
(2, 'Corrector Instant Age Rewind', 120, 5900.00, 5100.00, 2, 1),
(3, 'Polvo Compacto Stay Matte', 100, 4800.00, 4100.00, 3, 7),
(4, 'Rubor Baked Blush', 90, 5200.00, 4500.00, 4, 5),
(5, 'Iluminador MegaGlo', 110, 4300.00, 3700.00, 5, 7),
(6, 'Paleta de Sombras Nude', 80, 7800.00, 6900.00, 6, 11),
(7, 'Delineador Líquido Epic Ink', 130, 5000.00, 4300.00, 7, 3),
(8, 'Mascara Lash Princess', 160, 4700.00, 4000.00, 8, 8),
(9, 'Labial Matte Ink', 140, 5500.00, 4700.00, 9, 1),
(10, 'Gloss Shine Loud', 100, 5100.00, 4400.00, 10, 3),
(11, 'Set de Brochas Pro', 70, 9200.00, 8100.00, 11, 14),
(12, 'Esponja Blender', 200, 2500.00, 2000.00, 12, 19),
(13, 'Limpiador Facial Fresh', 85, 6400.00, 5600.00, 13, 2),
(14, 'Crema Corporal Hidratante', 75, 7100.00, 6200.00, 14, 2),
(15, 'Aceite Reparador Capilar', 65, 8900.00, 7800.00, 15, 20),
(16, 'Serum Vitamina C', 95, 8300.00, 7300.00, 16, 20),
(17, 'Primer Poreless Putty', 115, 6200.00, 5400.00, 17, 6),
(18, 'Spray Fijador Matte', 105, 5900.00, 5000.00, 18, 6),
(19, 'Removedor Bifásico', 125, 4600.00, 3900.00, 19, 4),
(20, 'Cosmetiquera Deluxe', 60, 8500.00, 7400.00, 20, 17),
(21, 'Tinta Labial Velvet', 90, 4900.00, 4200.00, 9, 3);
GO
```

-- CLIENTES

```
INSERT INTO Cliente (
    id_cliente, nombre, primer_apellido, segundo_apellido, telefono, email,
    direccion_exacta, codigo_mayorista, es_mayorista, id_canton
) VALUES
(1, 'Ana', 'Rodríguez', 'Mora', '88880001', 'ana1@gmail.com', 'Barrio centro, casa 1',
NULL, 0, 1),
```

(2, 'María', 'González', 'Vega', '88880002', 'maria2@gmail.com', 'Frente al parque', 'MAY-002', 1, 2),
 (3, 'Sofía', 'Herrera', 'López', '88880003', 'sofia3@gmail.com', 'Del súper 200 norte', NULL, 0, 3),
 (4, 'Valeria', 'Jiménez', 'Rojas', '88880004', 'valeria4@gmail.com', 'Residencial Las Flores', NULL, 0, 4),
 (5, 'Camila', 'Navarro', 'Solís', '88880005', 'camila5@gmail.com', 'Costado este de la escuela', 'MAY-005', 1, 5),
 (6, 'Isabella', 'Castro', 'León', '88880006', 'isabella6@gmail.com', 'Urbanización Los Arcos', NULL, 0, 6),
 (7, 'Daniela', 'Vargas', 'Cruz', '88880007', 'daniela7@gmail.com', 'Diagonal a la iglesia', 'MAY-007', 1, 7),
 (8, 'Lucía', 'Ramírez', 'Mena', '88880008', 'lucia8@gmail.com', 'Casa color azul', NULL, 0, 8),
 (9, 'Paula', 'Alfaro', 'Campos', '88880009', 'paula9@gmail.com', 'Condominio Vista Real', NULL, 0, 9),
 (10, 'Gabriela', 'Murillo', 'Arias', '88880010', 'gabriela10@gmail.com', '125 metros sur del banco', 'MAY-010', 1, 10),
 (11, 'Fernanda', 'Quesada', 'Pérez', '88880011', 'fernanda11@gmail.com', 'A la par de la pulpería', NULL, 0, 11),
 (12, 'Natalia', 'Monge', 'Cordero', '88880012', 'natalia12@gmail.com', 'Entrada principal del barrio', NULL, 0, 12),
 (13, 'Andrea', 'Rojas', 'Chacón', '88880013', 'andrea13@gmail.com', 'Apartamento 3B', 'MAY-013', 1, 13),
 (14, 'Carolina', 'Salas', 'Durán', '88880014', 'carolina14@gmail.com', 'Frente al colegio', NULL, 0, 14),
 (15, 'Elena', 'Mora', 'Arce', '88880015', 'elena15@gmail.com', 'Cerca de la clínica', NULL, 0, 15),
 (16, 'Tatiana', 'Céspedes', 'Ruiz', '88880016', 'tatiana16@gmail.com', 'Del puente 300 oeste', 'MAY-016', 1, 16),
 (17, 'Michelle', 'Paniagua', 'Soto', '88880017', 'michelle17@gmail.com', 'Urbanización El Sol', NULL, 0, 17),
 (18, 'Allison', 'Segura', 'Vargas', '88880018', 'allison18@gmail.com', 'Casa esquinera blanca', NULL, 0, 18),
 (19, 'Karla', 'Ureña', 'Mora', '88880019', 'karla19@gmail.com', 'Centro de Limón', 'MAY-019', 1, 19),
 (20, 'Jimena', 'Araya', 'Villalobos', '88880020', 'jimena20@gmail.com', '100 metros norte del parque', NULL, 0, 20);
 GO

-- PROMOCIONES

INSERT INTO Promocion (

id_promocion, nombre, tipo, descripcion, fecha_inicio, fecha_fin, activa

) VALUES

(1, 'Mayorista Enero', 'precio_mayorista', 'Precio mayorista para todos', '2024-01-10', '2024-01-31', 0),
(2, 'Combo Labiales', 'combo', 'Descuento en combo de labiales', '2024-02-01', '2024-02-20', 0),
(3, 'Promo Verano', 'otro', 'Promoción de temporada', '2024-03-01', '2024-03-31', 0),
(4, 'Mayorista Abril', 'precio_mayorista', 'Precios especiales de abril', '2024-04-05', '2024-04-25', 0),
(5, 'Combo Ojos', 'combo', 'Paleta más delineador', '2024-05-01', '2024-05-18', 0),
(6, 'Promo Mamá', 'otro', 'Descuentos por temporada', '2024-08-01', '2024-08-15', 0),
(7, 'Mayorista Septiembre', 'precio_mayorista', 'Mayorista abierto', '2024-09-01', '2024-09-30', 0),
(8, 'Combo Brochas', 'combo', 'Combo de brochas y esponjas', '2024-10-01', '2024-10-20', 0),
(9, 'Promo Noviembre', 'otro', 'Descuento general', '2024-11-10', '2024-11-30', 0),
(10, 'Black Week', 'otro', 'Promoción de noviembre', '2024-11-20', '2024-11-30', 0),
(11, 'Mayorista Enero 2025', 'precio_mayorista', 'Mayorista para todos', '2025-01-01', '2025-01-31', 0),
(12, 'Combo Piel', 'combo', 'Combo de cuidado facial', '2025-02-01', '2025-02-14', 0),
(13, 'Promo Marzo 2025', 'otro', 'Promoción especial', '2025-03-01', '2025-03-25', 0),
(14, 'Semana Santa', 'otro', 'Promoción de temporada', '2025-04-10', '2025-04-20', 0),
(15, 'Mayorista Julio', 'precio_mayorista', 'Precio mayorista especial', '2025-07-01', '2025-07-31', 0),
(16, 'Combo Cabello', 'combo', 'Productos para cuidado capilar', '2025-08-01', '2025-08-20', 0),
(17, 'Promo Octubre', 'otro', 'Descuento general', '2025-10-01', '2025-10-31', 0),
(18, 'Navidad 2025', 'otro', 'Promoción navideña', '2025-12-01', '2025-12-24', 0),
(19, 'Mayorista 2026', 'precio_mayorista', 'Mayorista temporal', '2026-01-15', '2026-02-15', 1),
(20, 'Combo Deluxe', 'combo', 'Combo de maquillaje premium', '2026-03-01', '2026-03-31', 1);

GO

-- RELACION PROMOCION-PRODUCTO

INSERT INTO PromocionProducto (id_promocion, id_producto) VALUES

(1,1),(1,2),(1,9),
(2,9),(2,10),
(3,5),(3,6),
(4,1),(4,17),
(5,6),(5,7),
(6,13),(6,14),
(7,3),(7,8),
(8,11),(8,12),

```
(9,4),(9,5),
(10,1),(10,9),(10,10),
(11,2),(11,17),
(12,13),(12,16),
(13,18),(13,19),
(14,6),(14,8),
(15,1),(15,3),
(16,15),(16,16),
(17,4),(17,18),
(18,20),(18,11),
(19,2),(19,9),
(20,6),(20,20);
GO
```

-- Inserción de datos de ventas

```
USE CatwalkGlowing_OLTP;
GO
```

```
DECLARE @i INT = 1;
DECLARE @fecha DATE;
DECLARE @id_cliente INT;
DECLARE @precio_final DECIMAL(10,2);
```

```
WHILE @i <= 500
BEGIN
```

```
    SET @fecha = DATEADD(DAY, (@i % 900), '2024-01-01');
    SET @id_cliente = ((@i - 1) % 20) + 1;
    SET @precio_final = 0.00;
```

```
    INSERT INTO Venta (id_venta, fecha, mensaje, precio_final, id_cliente)
    VALUES (
        @i,
        @fecha,
        CONCAT('Venta generada de prueba #', @i),
        @precio_final,
        @id_cliente
    );
```

```
    SET @i = @i + 1;
END;
GO
```

```
DECLARE @venta INT = 1;
DECLARE @id_detalle INT = 1;
```

```
DECLARE @producto1 INT;
DECLARE @producto2 INT;
DECLARE @cantidad1 INT;
DECLARE @cantidad2 INT;
DECLARE @precio1 DECIMAL(10,2);
DECLARE @precio2 DECIMAL(10,2);
DECLARE @tipo1 VARCHAR(20);
DECLARE @tipo2 VARCHAR(20);
DECLARE @promo1 INT;
DECLARE @promo2 INT;
```

```
WHILE @venta <= 500
BEGIN
    SET @producto1 = ((@venta - 1) % 20) + 1;
    SET @producto2 = ((@venta + 5) % 20) + 1;
```

```
    SET @cantidad1 = ((@venta - 1) % 4) + 1;
    SET @cantidad2 = ((@venta + 1) % 3) + 1;
```

```
    SET @tipo1 =
        CASE (@venta % 4)
            WHEN 0 THEN 'detalle'
            WHEN 1 THEN 'mayorista_volumen'
            WHEN 2 THEN 'mayorista_membresia'
            ELSE 'promocion'
        END;
```

```
    SET @tipo2 =
        CASE ((@venta + 1) % 4)
            WHEN 0 THEN 'detalle'
            WHEN 1 THEN 'mayorista_volumen'
            WHEN 2 THEN 'mayorista_membresia'
            ELSE 'promocion'
        END;
```

```
    SELECT
        @precio1 =
            CASE
                WHEN @tipo1 = 'detalle' THEN precio_detalle
```

```

        WHEN @tipo1 IN ('mayorista_volumen', 'mayorista_membresia') THEN
ISNULL(precio_mayorista, precio_detalle)
        WHEN @tipo1 = 'promocion' THEN ROUND(precio_detalle * 0.90, 2)
    END
FROM Producto
WHERE id_producto = @producto1;

```

```

SELECT
    @precio2 =
        CASE
            WHEN @tipo2 = 'detalle' THEN precio_detalle
            WHEN @tipo2 IN ('mayorista_volumen', 'mayorista_membresia') THEN
ISNULL(precio_mayorista, precio_detalle)
            WHEN @tipo2 = 'promocion' THEN ROUND(precio_detalle * 0.90, 2)
        END
FROM Producto
WHERE id_producto = @producto2;

```

```

    SET @promo1 = CASE WHEN @tipo1 = 'promocion' THEN ((@venta - 1) % 20) +
1 ELSE NULL END;

```

```

    SET @promo2 = CASE WHEN @tipo2 = 'promocion' THEN ((@venta + 3) % 20)
+ 1 ELSE NULL END;

```

```

INSERT INTO DetalleVenta (
    id_detalle, id_venta, id_producto, cantidad,
    precio_venta_unidad, tipo_precio, id_promocion
)
VALUES (
    @id_detalle, @venta, @producto1, @cantidad1,
    @precio1, @tipo1, @promo1
);

```

```

SET @id_detalle = @id_detalle + 1;

```

```

INSERT INTO DetalleVenta (
    id_detalle, id_venta, id_producto, cantidad,
    precio_venta_unidad, tipo_precio, id_promocion
)
VALUES (
    @id_detalle, @venta, @producto2, @cantidad2,
    @precio2, @tipo2, @promo2
);

```

```

SET @id_detalle = @id_detalle + 1;

```

```

    SET @venta = @venta + 1;
END;
GO

UPDATE V
SET V.precio_final = X.total_venta
FROM Venta V
INNER JOIN (
    SELECT
        id_venta,
        SUM(cantidad * precio_venta_unidad) AS total_venta
    FROM DetalleVenta
    GROUP BY id_venta
) X ON V.id_venta = X.id_venta;
GO

```

Script 8: creación base DW

```

CREATE DATABASE CatwalkGlowing_DW;
GO

USE CatwalkGlowing_DW;
GO

-- =====
-- LIMPIEZA PREVIA SI LAS TABLAS YA EXISTEN
-- =====

IF OBJECT_ID('dbo.StgVentaDetalle', 'U') IS NOT NULL
    DROP TABLE dbo.StgVentaDetalle;
GO

IF OBJECT_ID('dbo.StgTipoDePrecio', 'U') IS NOT NULL
    DROP TABLE dbo.StgTipoDePrecio;
GO

IF OBJECT_ID('dbo.StgTiempo', 'U') IS NOT NULL
    DROP TABLE dbo.StgTiempo;
GO

IF OBJECT_ID('dbo.StgPromocion', 'U') IS NOT NULL
    DROP TABLE dbo.StgPromocion;

```

GO

```
IF OBJECT_ID('dbo.StgProducto', 'U') IS NOT NULL
    DROP TABLE dbo.StgProducto;
```

GO

```
IF OBJECT_ID('dbo.StgCliente', 'U') IS NOT NULL
    DROP TABLE dbo.StgCliente;
```

GO

```
-- =====
-- STAGING DE CLIENTES
-- Fuente OLTP:
--  Cliente, Canton, Provincia
-- Destino final:
--  DimCliente
-- =====
```

```
CREATE TABLE dbo.StgCliente (
    id_cliente INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    primer_apellido VARCHAR(100) NOT NULL,
    segundo_apellido VARCHAR(100) NULL,
    nombre_completo VARCHAR(300) NULL,
    canton VARCHAR(100) NOT NULL,
    provincia VARCHAR(100) NOT NULL,
    fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
);
GO
```

```
-- =====
-- STAGING DE PRODUCTOS
-- Fuente OLTP:
--  Producto, Categoria, Marca
-- Carga: CDC incremental
-- Destino final:
--  DimProducto
-- =====
```

```
CREATE TABLE dbo.StgProducto (
    id_producto INT NOT NULL,
    nombre_producto VARCHAR(100) NOT NULL,
```

```

        categoria VARCHAR(100) NOT NULL,
        marca VARCHAR(100) NOT NULL,
        fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
    );
GO

```

```

-- =====
-- STAGING DE PROMOCIONES
-- Fuente OLTP:
--   Promocion
-- Destino final:
--   DimPromocion
-- =====

```

```

CREATE TABLE dbo.StgPromocion (
    id_promocion INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    tipo VARCHAR(50) NOT NULL,
    descripcion VARCHAR(255) NULL,
    fecha_inicio DATE NULL,
    fecha_fin DATE NULL,
    fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
);
GO

```

```

-- =====
-- STAGING DE TIEMPO
-- Fuente OLTP:
--   Venta.fecha
-- Destino final:
--   DimTiempo
-- =====

```

```

CREATE TABLE dbo.StgTiempo (
    id_tiempo INT NOT NULL,
    fecha DATE NOT NULL,
    dia INT NOT NULL,
    mes INT NOT NULL,
    nombre_mes VARCHAR(20) NOT NULL,
    trimestre INT NOT NULL,
    anio INT NOT NULL,
    nombre_dia VARCHAR(20) NOT NULL,

```

```
    fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
);
GO
```

```
-- =====
-- STAGING DE TIPO DE PRECIO
-- Fuente OLTP:
-- Producto.precio_detalle, Producto.precio_mayorista
-- Destino final:
-- DimTipoDePrecio
-- =====
```

```
CREATE TABLE dbo.StgTipoDePrecio (
    id_tipo_de_precio INT NOT NULL,
    precio_detalle DECIMAL(10,2) NOT NULL,
    precio_mayorista DECIMAL(10,2) NULL,
    fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
);
GO
```

```
-- =====
-- STAGING DE VENTAS Y DETALLE
-- Fuente OLTP:
-- Venta, DetalleVenta, Producto (para tipo de precio)
-- Carga: CDC incremental
-- Destino final:
-- FactVenta
-- =====
```

```
CREATE TABLE dbo.StgVentaDetalle (
    id_venta INT NOT NULL,
    id_detalle INT NOT NULL,
    fecha DATE NOT NULL,
    id_cliente INT NOT NULL,
    id_producto INT NOT NULL,
    id_promocion INT NULL,
    id_tipo_de_precio INT NOT NULL,
    cantidad_vendida INT NOT NULL,
    precio_aplicado DECIMAL(10,2) NOT NULL,
    monto_total DECIMAL(10,2) NOT NULL,
    tipo_precio_aplicado VARCHAR(50) NOT NULL,
    fecha_carga DATETIME NOT NULL DEFAULT GETDATE()
);
```

```
);  
GO
```

```
-- =====  
-- TABLA DE ESTADO PARA CDC MANUAL  
-- Almacena el ultimo LSN procesado  
-- por cada tabla con carga incremental  
-- =====
```

```
CREATE TABLE dbo.cdc_state (  
    table_name NVARCHAR(100) NOT NULL PRIMARY KEY,  
    last_lsn BINARY(10) NOT NULL  
);  
GO
```

```
INSERT INTO dbo.cdc_state (table_name, last_lsn) VALUES ('Venta',  
0x00000000000000000000);  
INSERT INTO dbo.cdc_state (table_name, last_lsn) VALUES ('Producto',  
0x00000000000000000000);  
GO
```

```
CREATE INDEX IX_StgCliente_id_cliente  
ON dbo.StgCliente(id_cliente);  
GO
```

```
CREATE INDEX IX_StgProducto_id_producto  
ON dbo.StgProducto(id_producto);  
GO
```

```
CREATE INDEX IX_StgPromocion_id_promocion  
ON dbo.StgPromocion(id_promocion);  
GO
```

```
CREATE INDEX IX_StgTiempo_fecha  
ON dbo.StgTiempo(fecha);  
GO
```

```
CREATE INDEX IX_StgTipoDePrecio_id  
ON dbo.StgTipoDePrecio(id_tipo_de_precio);  
GO
```

```
CREATE INDEX IX_StgVentaDetalle_id_venta  
ON dbo.StgVentaDetalle(id_venta);
```

GO

```
CREATE INDEX IX_StgVentaDetalle_id_cliente  
ON dbo.StgVentaDetalle(id_cliente);  
GO
```

```
CREATE INDEX IX_StgVentaDetalle_id_producto  
ON dbo.StgVentaDetalle(id_producto);  
GO
```

-- Dimensiones y Fact Table

```
CREATE TABLE DimTiempo (  
    id_tiempo INT PRIMARY KEY,  
    fecha DATE NOT NULL,  
    dia INT NOT NULL,  
    nombre_mes VARCHAR(20) NOT NULL,  
    trimestre INT NOT NULL,  
    anio INT NOT NULL,  
    nombre_dia VARCHAR(20) NOT NULL  
);
```

```
CREATE TABLE DimCliente (  
    id_cliente INT PRIMARY KEY,  
    nombre_completo VARCHAR(300) NOT NULL,  
    canton VARCHAR(100) NOT NULL,  
    provincia VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE DimProducto (  
    id_producto INT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    categoria VARCHAR(50) NOT NULL,  
    marca VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE DimTipoDePrecio (  
    id_tipo_de_precio INT PRIMARY KEY,  
    precio_detalle DECIMAL(10,2) NOT NULL,  
    precio_mayorista DECIMAL(10,2) NULL  
);
```

```
CREATE TABLE DimPromocion (  
    id_promocion INT PRIMARY KEY,
```

```

nombre VARCHAR(100) NOT NULL,
tipo VARCHAR(20) NOT NULL
    CHECK (tipo IN ('precio_mayorista', 'combo', 'otro', 'ninguna')),
descripcion VARCHAR(255),
fecha_inicio DATE NULL,
fecha_fin DATE NULL
);

INSERT INTO DimPromocion (id_promocion, nombre, tipo, descripcion, fecha_inicio,
fecha_fin)
VALUES (0, 'Sin promoción', 'ninguna', NULL, NULL, NULL);

CREATE TABLE FactVenta (
    id_fact INT IDENTITY(1,1) PRIMARY KEY,
    id_tiempo INT NOT NULL,
    id_cliente INT NOT NULL,
    id_producto INT NOT NULL,
    id_promocion INT NOT NULL DEFAULT 0,
    id_tipo_de_precio INT NOT NULL,
    cantidad_vendida INT NOT NULL,
    monto_total DECIMAL(10,2) NOT NULL,
    tipo_precio_aplicado VARCHAR(20) NOT NULL
    CHECK (tipo_precio_aplicado IN ('detalle', 'mayorista')),
    FOREIGN KEY (id_tiempo) REFERENCES DimTiempo(id_tiempo),
    FOREIGN KEY (id_cliente) REFERENCES DimCliente(id_cliente),
    FOREIGN KEY (id_producto) REFERENCES DimProducto(id_producto),
    FOREIGN KEY (id_promocion) REFERENCES DimPromocion(id_promocion),
    FOREIGN KEY (id_tipo_de_precio) REFERENCES
DimTipoDePrecio(id_tipo_de_precio)
);

```

Debe suministrar los archivos correspondientes al proyecto/solución, y además la documentación de las pruebas realizadas para verificar las cargas incrementales.